TXFETCH

Full Technical Documentation

Architecture · Code Walkthrough · Scoring Algorithm · Per-network Breakdown

by xaynov (@ownstates)

1. Overview

TXFetch is a command-line utility for searching blockchain transactions using indirect evidence. It solves the reverse problem: instead of knowing the transaction hash and looking up its data, the user knows the data and wants to find the hash.

Typical use case: a person knows they sent or received a transfer at approximately a certain time for a certain amount, but the transaction hash was lost or is unknown. TXFetch queries public blockchain APIs, filters the results, scores each candidate and returns a ranked list.

Network	Coins	API
TRON	USDT, USDC	TronGrid Events API
TON	TON, USDT, NOT	TONAPI / TONCenter v3
ETH	USDT, USDC, ETH	Etherscan
BSC	USDT, USDC, BNB	BscScan
Base	USDC, ETH	Basescan
Solana	SOL, USDC, USDT, RAY, JUP	Public RPC / Helius
Bitcoin	BTC	mempool.space
Litecoin	LTC	litecoinspace.org
Dogecoin	DOGE	dogechain
Bitcoin Cash	BCH	blockchair

---

2. Architecture

The project is built on three patterns: a single entry point, the adapter pattern for networks, and a shared scoring module.

```
txfetch/
├── main.py          # entry point — TUI, argparse, search pipeline
├── pyproject.toml   # package config and CLI entry points
├── requirements.txt
├── .env.example
└── modules/
    ├── models.py    # TXQuery, TXCandidate, score_candidate,
amount_matches
    ├── config.py    # api keys and constants from .env
    ├── matcher.py   # universal filter and scorer
    ├── tron.py      # TRON adapter
    ├── ton.py       # TON adapter
    ├── evm_base.py  # base class for EVM networks
    ├── eth.py       # Ethereum
```

```
├── bsc.py       # BNB Smart Chain
├── base_chain.py# Base
├── solana.py    # Solana adapter
├── utxo_base.py # base class for UTXO networks
├── btc.py       # Bitcoin
├── ltc.py       # Litecoin
├── doge.py      # Dogecoin
└── bch.py       # Bitcoin Cash
```

2.1 Data flow

User inputs parameters → `TXQuery` → adapter queries API → `matcher.find_matches()` → `TXCandidate` list sorted by confidence.

2.2 Patterns

Adapter pattern

Every adapter implements one method — fetch_candidates(query) → list[TXCandidate]. main.py has no knowledge of the internal differences between networks. Adding a new network requires a new file and one line in the adapter dispatch table.

Base class pattern

EVMBase contains all shared logic for Ethereum-compatible networks: timestamp-to-block conversion, tokentx requests, pagination. ETH, BSC and Base inherit it and only add config (URL, tokens, API key).

UTXOBase contains shared logic for Bitcoin-like networks: block height estimation, block pagination, vout/vin/OP_RETURN parsing. BTC, LTC, DOGE and BCH inherit it.

Dispatch table

Instead of a long if/elif chain — an adapter dictionary. importlib dynamically imports the needed module by network key. Adding a network does not require changing run_search logic.

---

3. Code Walkthrough

A plain-language explanation of each file — what it does, why it exists, and how it connects to the rest of the project.

main.py — entry point

This is the first file that runs. It is responsible for three things: showing the interface, collecting parameters from the user, and launching the right adapter to display results.

Two operating modes. If the user runs txf with no arguments — a TUI opens (prompt_toolkit): ASCII art, arrow-key network menu, step-by-step input. If flags are passed (txf -n tron -c USDT ...) — argparse collects them and runs the search immediately without the interface.

Key functions:

- select_network() — arrow-key network menu via prompt_toolkit.Application
- ask_params() — step-by-step input: date, time, range menu, coin, amount, memo
- run_search() — builds TXQuery, calls the adapter via dispatch table, handles memo fallback
- _print_results() — prints result table, offers TXID copy and Excel export

> i memo fallback: if memo is specified but no results are found — automatically retries without it and notifies the user.

models.py — shared data types

Contains three entities used everywhere: TXQuery, TXCandidate and scoring functions.

TXQuery is the query object. main.py assembles it from user input and passes it to the adapter. Contains: time window (time_from / time_to in UTC), coin, optional amount (or range amount..amount_to), tolerance and memo.

TXCandidate is a search result. All adapters return a list of these objects, so main.py works the same way for every network. Contains: transaction hash, timestamp, amount, addresses, memo and confidence.

amount_matches() — a dedicated function for amount checking. Supports two modes: exact match with tolerance, and range mode (if amount_to is set). Used inside score_candidate.

score_candidate() — computes confidence from three criteria: time, amount, memo. If amount is specified but does not match — immediately returns 0.0 without further computation.

config.py — configuration

Reads the .env file from the project root on startup and exposes values as named constants. No external dependencies — pure Python.

Two categories: API keys (TRONGRID_API_KEY, ETHERSCAN_API_KEY, etc.) and runtime constants (REQUEST_TIMEOUT, PAGE_LIMIT, MAX_PAGES, PAGE_DELAY, MIN_CONFIDENCE). All have default values — if a key is not set, the adapter works in public mode. If a constant is not set — the default is used.

> i config also reads from os.environ — keys can be passed as environment variables without a .env file.

matcher.py — filtering and scoring

Single public function: find_matches(events, query, parser, network, min_confidence).

Accepts raw API events, a parser function (how to extract fields from the specific format), the query and the network name. Internally: for each event calls parser → gets (timestamp, amount, memo, from, to, txid) → calls score_candidate → discards below threshold → sorts descending.

Why a separate module: previously every adapter copied the same filter loop. After refactoring, scoring logic lives in one place and adapters only define a parser — how to extract fields from their specific API format.

tron.py — TRON adapter

The simplest adapter. TronGrid supports min/max_block_timestamp on the contract events endpoint — the server does the time filtering itself. The adapter only paginates via fingerprint cursor and passes events to matcher.

Notable: on request failure, retries up to 3 times with a delay before stopping. Warns the user if results may be incomplete.

ton.py — TON adapter

Operates in two modes depending on the coin. For Jetton (USDT, NOT) — TONAPI /jettons/{id}/transfers with start_date/end_date. For native TON — TONCenter v3 /transactions with start_utime/end_utime.

TON is the only network where memo is a native protocol field. The transfer comment is stored in message_content.decoded.comment and extracted automatically. This is not a workaround — it is a standard blockchain capability.

Pagination via before_lt — logical time of the last record. This is not a Unix timestamp but an internal TON sequence number that guarantees a unique position in the chain's history.

evm_base.py + eth.py / bsc.py / base_chain.py — EVM adapters

EVMBase contains all shared logic. Timestamp-to-block conversion (two getblocknobytime calls), tokentx request by block range, page/offset pagination. Each concrete network is 10-15 lines of config: URL, API key, token dictionary with addresses and decimals.

> ⚠ BSC USDT uses 18 decimals, not 6 as on Ethereum. EVMBase applies the correct value automatically from the token config.

solana.py — Solana adapter

The most complex adapter due to Solana's architecture. No global endpoint for time-based filtering — the token mint address is used as the entry point.

Two modes depending on whether a Helius key is available. Without key: getSignaturesForAddress(mint) → filter by blockTime → getTransaction for each signature → parse preTokenBalances/postTokenBalances delta. With Helius key: /v0/transactions accepts a list of signatures and returns ready-to-use tokenAmount, fromUser, toUser, memo fields — much faster.

Memo is extracted with dual verification: exact programId of the Memo program (Mem_oMoDT32...) and a fuzzy 'memo' name check as fallback. Searched in both main instructions and meta.innerInstructions — the latter is important for DEX transactions where the transfer happens inside another program.

utxo_base.py + btc.py / ltc.py / doge.py / bch.py — UTXO adapters

UTXOBase contains shared logic for Bitcoin-like networks. Key parameter BLOCK_TIME — average block time in seconds (600 for BTC, 150 for LTC, 60 for DOGE). Used to estimate block height from timestamp: tip_height - (now - target_ts) / BLOCK_TIME.

Transaction amount is the sum of all non-OP_RETURN outputs (vout). Includes sender change. This is an imprecision of the UTXO model — a wider tolerance is recommended when searching BTC transactions by amount.

Memo via OP_RETURN: a special output type where some services embed arbitrary text. Decoded from hex to UTF-8. Absent in most standard transfers.

---

4. Scoring algorithm

After receiving transactions from the API, each candidate is assigned a confidence score from 0 to 1. Candidates below the threshold are discarded, the rest are sorted descending.

Criterion	Weight	Logic
Time	0.5	Linear from time_from. TX at 20:38:00 → 1.0, TX at 20:38:59 → ~0.0
Amount	0.4	Exact match → 1.0. Within tolerance → linear. Outside → immediate rejection.
Memo	0.1	Case-insensitive substring match → 1.0. No match → 0.0.

4.1 Formula

```
confidence = 0.5 × time_score + 0.4 × amount_score + 0.1 × memo_score
```

4.2 Amount range

If the user enters 500-800 instead of a single number — range mode is activated. A candidate passes if amount_from ≤ event_amount ≤ amount_to. The score decreases linearly from the center toward the edges — a TX exactly at the midpoint gets 1.0.

4.3 Weight redistribution

If a criterion is not specified — its weight is redistributed to the others:

- No amount and no memo → confidence = time_score
- No amount, memo present → 0.9 × time + 0.1 × memo
- Amount present, no memo → weights 0.5 and 0.4 are normalized proportionally

4.4 Memo fallback

If the user specified memo but the search returned no results — main.py automatically creates a copy of the query without memo and retries. The user sees a warning. This prevents missing results when the sender did not add a comment.

---

5. Per-network breakdown

TRON

Parameter	Value
Time filter	min/max_block_timestamp server-side (milliseconds)
Amount units	sun: 1 USDT = 1,000,000 sun (decimals=6)
Memo	not supported in standard TRC-20 Transfer
Pagination	fingerprint cursor from response meta field
Limitations	rate limit without key ~1 req/sec, 429 on second page

TON

Parameter	Value
Jetton filter	start_date / end_date (Unix seconds) server-side
Native filter	start_utime / end_utime via TONCenter v3
Amount units	nanoTON for TON (decimals=9), USDT decimals=6, NOT decimals=9
Memo	native field in_msg.message_content.decoded.comment
Pagination	before_lt (logical time) for Jetton, offset for native TON
Limitations	two different APIs for Jetton and native TON

ETH / BSC / Base

Parameter	Value
Time filter	timestamp → block via getblocknobytime, two calls
Request	tokentx?contractaddress=...&startblock=X&endblock=Y (no address)
ETH USDT decimals	6
BSC USDT decimals	18 (different from Ethereum!)
Memo	not supported in ERC-20 Transfer
Pagination	page / offset, up to 1000 records per page
Limitations	API key needed for stable use, 5 req/sec on free tier

Solana

Parameter	Value
Approach	getSignaturesForAddress(mint) → filter by blockTime → getTransaction
With Helius	/v0/transactions — returns tokenAmount, fromUser, toUser, memo ready-to-use
Amount units	uiAmount from preTokenBalances/postTokenBalances delta
Memo	Memo Program (Mem_oMoDT32...) in instructions and innerInstructions
DEX transfers	searched in meta.innerInstructions for transactions via DEX
Limitations	public RPC heavily rate-limited, Helius key strongly recommended

Bitcoin / Litecoin / Dogecoin / Bitcoin Cash

Parameter	Value
Approach	estimate block height → iterate blocks → iterate TXs inside block
BLOCK_TIME	BTC: 600s, LTC: 150s, DOGE: 60s, BCH: 600s
Amount units	satoshi: sum of all non-OP_RETURN vout / SATOSHI
Memo	OP_RETURN field, hex → UTF-8. Absent in most standard transfers.
Pagination	25 TX per call via start_index
Limitations	no server-side filter by time or amount — all filtering is local

> ⚠ Bitcoin amount includes sender change. Use a wider tolerance (0.1 or more) when searching BTC transactions by amount.